# Introduction

Java is one of the most widely used and powerful programming languages in the field of computer science. Developed by Sun Microsystems and later acquired by Oracle Corporation, Java is a high-level, object-oriented programming language that follows the principle of "Write Once, Run Anywhere" (WORA). This feature is possible due to the Java Virtual Machine (JVM), which allows Java programs to run on any platform that supports the JVM, making it a platform-independent language. Over the years, Java has evolved into a robust tool for building a wide range of applications—from desktop software and web applications to mobile apps and enterprise-level systems.

The primary objective of this assignment is to explore and implement fundamental Java programming concepts such as classes and objects, inheritance, polymorphism, abstraction, exception handling, file operations, and data structures like arrays or collections. Through this assignment, students gain hands-on experience in writing clean, modular, and efficient code that adheres to object-oriented design principles. Java's extensive library support and built-in functionalities provide a solid foundation for learning best programming practices and improving problem-solving abilities.

In this assignment, a particular problem or real-life scenario has been modeled using appropriate classes and methods. Emphasis is placed on writing readable and maintainable code using comments, indentation, and proper naming conventions. The assignment also includes the algorithm or logic behind the implementation, input-output structure, and a discussion of how the program works. Additional features such as user interaction through console input, exception handling for robustness, and file input/output (if applicable) have been integrated to reflect practical software development techniques.

Overall, this Java assignment not only strengthens the theoretical understanding of programming concepts but also helps in developing critical thinking and debugging skills that are essential for any aspiring computer science professional. It lays the groundwork for advanced study in Java frameworks like Spring Boot, JavaFX, and Android development, thereby bridging the gap between academic learning and real-world software engineering.

# Loops in Java

Loops in Java are fundamental control structures that enable repeated execution of a block of code based on a specific condition. Rather than writing the same code multiple times, loops allow programmers to automate repetitive tasks efficiently and elegantly. Mastering loops is crucial for building logical constructs such as data processing, pattern generation, iterative calculations, and more.

Java provides three primary types of loops, each tailored to different use cases:

**Types of Loops in Java**

- **`for` loop** – Used when the number of iterations is predetermined. Ideal for counter-controlled loops.
- **`while` loop** – Condition is evaluated before each iteration. Suitable when the number of iterations is not known in advance.
- **`do-while` loop** – Executes the loop body at least once, since the condition is checked after the loop body.

**Example: Sum of First 5 Natural Numbers Using `for` Loop**

Listing 1: Java Program to Find Sum of First 5 Natural Numbers

```
public class LoopExample {
    public static void main(String[] args) {
        int sum = 0;
        for (int i = 1; i <= 5; i++) {
            sum += i;
        }
        System.out.println("Sum=" + sum);
    }
}
```

**Output:** `Sum = 15`

# Function Overloading in Java

Function Overloading in Java is a feature that allows a class to have more than one method with the same name, but with different parameter lists. It enhances readability and reusability of code and is a part of **compile-time (static) polymorphism**.

Overloaded methods must differ in the **number**, **type**, or **order** of parameters. Return type alone is not sufficient for method overloading.

## Rules of Function Overloading

- Same method name but different parameter list.
- Method can differ by:
  - Number of parameters
  - Data type of parameters
  - Order of parameters
- Return type can be different but it's not used to distinguish overloaded methods.

**Example: Function Overloading with Varying Parameters**

Listing 2: Java Program Demonstrating Function Overloading

```
public class Calculator {
 public int add(int a, int b) {
        return a + b;}public int add(int a, int b, int c) {return a + b +c;
    }public double add(double a, double b) {return a + b;
    }public static void main(String[] args) { Calculator calc = new Calculator();
        System.out.println("Sum␣of␣2␣ints:␣" + calc.add(5, 10));
        System.out.println("Sum␣of␣3␣ints:␣" + calc.add(5, 10, 15));
        System.out.println("Sum␣of␣2␣doubles:␣" + calc.add(5.5, 4.5)); }}
```

## Output:

```
Sum of 2 ints: 15
Sum of 3 ints: 30
Sum of 2 doubles: 10.0
```

# Arrays in Java

Arrays in Java are fundamental data structures that allow storage of multiple values of the same data type under a single variable name. They are useful for efficiently handling collections of data such as lists of numbers, names, or objects, and they simplify iteration and processing of related values. Each element in the array is identified by its index, starting from 0.

**Key Features of Arrays in Java**

- Arrays are indexed collections of fixed size.
- The index starts from 0 and goes up to (length - 1).
- Arrays can store primitive data types and objects.
- Declared with square brackets `[]`.

**Example: Storing and Printing Elements of an Integer Array**

Listing 3: Java Program to Print Elements of an Array

```
public class ArrayExample {
    public static void main(String[] args) {
        int[] numbers = {10, 20, 30, 40, 50};
        System.out.println("Array elements are:");
        for (int i = 0; i < numbers.length; i++) {
            System.out.println("Element atindex" + i + ": " + numbers[i]);
        }
    }
}
```

## Output:

```
Array elements are:
Element at index 0: 10
Element at index 1: 20
Element at index 2: 30
Element at index 3: 40
Element at index 4: 50
```

# OOP Concepts in Java

Object-Oriented Programming (OOP) is a paradigm that organizes software design around data, or objects, rather than functions and logic. Java is a pure object-oriented language (except for primitive types) and supports all major OOP principles. Mastering OOP helps in building scalable, reusable, and maintainable applications.

## Four Pillars of OOP in Java

- **Encapsulation** – Binding data and code together within a class and restricting direct access to some components.
- **Inheritance** – Mechanism by which one class acquires the properties and behavior (methods) of another class.
- **Polymorphism** – Ability of a variable, function, or object to take multiple forms.
- **Abstraction** – Hiding complex implementation details and showing only the essential features.

**Example: Demonstrating Encapsulation in Java**

Listing 4: Java Program to Demonstrate Encapsulation

```java
class Student {
    private String name;  // Private = Encapsulation

    // Getter method
    public String getName() {
        return name;  }
    // Setter method
    public void setName(String newName) {
        name = newName; }}
public class Main {
    public static void main(String[] args) {
        Student s1 = new Student();
        s1.setName("Alice");
        System.out.println("Student Name: " + s1.getName());}}
```

**Output:** `Student Name:  Alice`

# Threading in Java

Threading in Java allows a program to perform multiple tasks concurrently, improving the efficiency and responsiveness of applications. Java provides built-in support for multithreading through the Thread class and the Runnable interface. These enable the creation and management of threads that can execute independently or cooperatively, depending on the design of the application.

A thread is a lightweight sub-process and the smallest unit of CPU execution. Unlike processes, threads share the same memory space, which allows for easy communication between them, but also introduces challenges such as synchronization and thread safety.

Multithreading is especially beneficial in situations such as:

Animations and GUI applications, where user interface responsiveness is critical.

File I/O operations, where reading and writing files can occur in the background without blocking the main thread.

Parallel computations, such as sorting large datasets or performing mathematical calculations concurrently.

Network operations, where waiting for data from remote sources can be handled asynchronously.

In Java, you can create a thread in two primary ways:

Extending the Thread class and overriding its run() method.

Implementing the Runnable interface, passing an instance of the class to a Thread object.

Additionally, Java provides high-level concurrency utilities in the javautil package, including ExecutorService, Callable, Future, and synchronization tools like locks, semaphores, and countdown latches.

Proper use of threading can greatly improve application performance, but it must be handled with care to avoid issues such as:

Race conditions

Deadlocks

Thread starvation

By understanding and applying multithreading principles effectively, developers can build robust, efficient, and responsive Java applications.

## Key Concepts in Java Threading

- **Thread Class** – Inherit the `Thread` class and override the `run()` method.
- **Runnable Interface** – Implement `Runnable` and pass the object to a `Thread`.
- **start()** – Starts a new thread which calls the `run()` method internally.
- **run()** – Contains the code that should execute in a separate thread.

### Example: Creating Threads by Extending `Thread` Class

Listing 5: Java Program to Create Threads Using Thread Class

```
public void run() {
for (int i = 1; i <= 5; i++) {
System.out.println(Thread.currentThread().getName() + " - Count: " + i);try {
    Thread.sleep(500);
// pause for 0.5 seconds
            }
            catch (InterruptedException e) {System.out.println(e);}} }}
    public class ThreadDemo {

    public static void main(String[] args) {
        MyThread t1 = new MyThread();

        MyThread t2 = new MyThread();

        t1.setName("Thread A");t2.setName("Thread B");t1.start();t2.start();
        }
        }
```

## Sample Output:

```
Thread A - Count: 1
Thread B - Count: 1
Thread A - Count: 2
Thread B - Count: 2
```

# JDBC in Java

JDBC (Java Database Connectivity) is an API that allows Java programs to interact with relational databases. Using JDBC, Java applications can connect to a database, send SQL queries, and retrieve and manipulate data in a standardized way.

JDBC is part of the Java Standard Edition platform and works with all major databases like MySQL, Oracle, PostgreSQL, and SQLite.

**Steps to Use JDBC in Java**

1. **Import JDBC packages:** `import java.sql.*;`
2. **Load JDBC driver:** `Class.forName("com.mysql.cj.jdbc.Driver");`
3. **Establish connection:** `Connection con = DriverManager.getConnection(...);`
4. **Create statement:** `Statement stmt = con.createStatement();`
5. **Execute query:** `ResultSet rs = stmt.executeQuery(...);`
6. **Process results:** Use `while(rs.next())` to read data.
7. **Close connection:** `con.close();`

### Example: Connecting to MySQL and Fetching Data

Listing 6: Java JDBC Program to Connect to MySQL

```java
import java.sql.*;

public class JDBCExample {
    public static void main(String[] args) {
        try {
            // Step 1: Load Driver
            Class.forName("com.mysql.cj.jdbc.Driver");

            // Step 2: Establish connection
            Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/mydatabase", "root", "password");

            // Step 3: Create statement
            Statement stmt = con.createStatement();

            // Step 4: Execute query
            ResultSet rs = stmt.executeQuery("SELECT␣*␣FROM␣students");

            // Step 5: Process results
            while (rs.next()) {
                System.out.println("ID:␣" + rs.getInt("id") +
                                   ",␣Name:␣" + rs.getString("name"));
            }

            // Step 6: Close connection
            con.close();

        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

## Note:

- Make sure to include the MySQL JDBC driver (e.g., `mysql-connector-java.jar`) in your project classpath.
- Replace database name, username, password, and table name with actual values.

## Problem Statement-1:-

**Write a program in java that sorts half of element in ascending and rest half of the elements in descending order.**

### ❒ Objective:-

- **Input Handling:** To write a Java program that takes input for an array of integers from the user. The program should be able to handle any valid size of the array entered by the user.
- **Array Splitting:** To divide the array into two halves based on its length. If the array has an odd number of elements, the extra element belongs to the second half.
- **Ascending Sort:** To sort the first half of the array in ascending order using Java's built-in sorting method. This ensures that the initial part of the array is organized from smallest to largest.
- **Descending Sort:** To sort the second half of the array in descending order. This requires converting the elements to an object array to use reverse order sorting.
- **Combining Halves:** To combine both sorted halves back into the original array. Care should be taken to maintain the order of each sorted half during merging.
- **Output Display:** To display the final array with the first half sorted in ascending order and the second half in descending order as the final output to the user.

## ❒ Algorithm:-

**Algorithm 1** Half Sort Array (First Half Ascending, Second Half Descending)

START
INPUT THE SIZE OF THE ARRAY $N$
DECLARE AN INTEGER ARRAY $ARR[N]$
**for** $I = 0$ TO $N - 1$ **do**
  INPUT $ARR[I]$
**end for**
CALCULATE MIDPOINT $MID = N/2$
SORT THE FIRST HALF OF THE ARRAY $ARR[0$ TO $MID - 1]$ IN ASCENDING ORDER
CREATE A TEMPORARY INTEGER ARRAY $SECONDHALF[N - MID]$
**for** $I = MID$ TO $N - 1$ **do**
  $SECONDHALF[I - MID] \leftarrow ARR[I]$
**end for**
SORT $SECONDHALF$ IN DESCENDING ORDER
**for** $I = MID$ TO $N - 1$ **do**
  $ARR[I] \leftarrow SECONDHALF[I - MID]$
**end for**
OUTPUT THE MODIFIED ARRAY $ARR$
STOP

## ❒ Code:-

```
import java.util.Arrays;

import java.util.Collections;

import java.util.Scanner;

public class HalfSortArray {

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        // Take array size input
        System.out.print("Enter the number of elements: ");
        int n = scanner.nextInt();
        int[] arr = new int[n];
        // Take array elements input
        System.out.println("Enter the elements:");
        for (int i = 0; i < n; i++) {
            arr[i] = scanner.nextInt();
```

```
        }

        int mid = n / 2;

        // Sort first half in ascending order

        Arrays.sort(arr, 0, mid);

        // Copy second half to Integer[] for reverse sorting
        Integer[] secondHalf = new Integer[n - mid];

        for (int i = mid; i < n; i++) {
            secondHalf[i - mid] = arr[i];
        }

        // Sort second half in descending order

        Arrays.sort(secondHalf, Collections.reverseOrder());

        // Copy back to original array

        for (int i = mid; i < n; i++) {
            arr[i] = secondHalf[i - mid];
        }

        // Print the result
        System.out.println("Array after sorting

        half ascending and half descending:");

        for (int num : arr) {
            System.out.print(num + " ");
        }
        scanner.close();
    }
}
```

## ❒ Output:-

```
"C:\Program Files\Java\jdk-23\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Community Edition 2024.3.1\lib
Enter the number of elements: 5
Enter the elements:
5
7
9
5
Array after sorting half ascending and half descending:
1 5 9 7 5
Process finished with exit code 0
```

**Figure 1**

## ❒ Conclusion:-

The Java program successfully sorts the first half of the array in ascending order and the second half in descending order based on user input. It demonstrates the effective use of array manipulation, Java's built-in `Arrays.sort()` method, and `Collections.reverseOrder()`. This task strengthens the understanding of partial array sorting and highlights the importance of handling primitive and wrapper types in Java. It also reinforces logical thinking and modular programming practices.

## Problem Statement-2:-

**Write a program in java that accepts a 2D matrix and prints the matrix with row minimum and column minimum values.**
**4 3 5 3**
**1 0 7 0**
**8 4 6 4**
**1 0 5**

### ❒ Objective:-

- **Matrix Input:** To develop a Java program that reads a 2D matrix from the user. The program should accept user-defined numbers of rows and columns.
- **Storing Data:** To store the entered matrix elements into a 2D array structure. This allows further processing for analysis and output.
- **Row Minimums:** To compute the minimum element from each row of the matrix. These values will be displayed alongside each respective row.
- **Column Minimums:** To determine the smallest value from each column in the matrix. These values will be printed separately after the matrix.
- **Formatted Output:** To display the matrix elements clearly in a tabular form. Row minimums are shown at the end of each row for readability.
- **Minimum Summary:** To provide a summary of column minimums below the matrix. This ensures the output reflects both row-wise and column-wise analysis.

## ❒ Algorithm:-

**Algorithm 2** Find Row and Column Minimums in a Matrix

```
START
INPUT NUMBER OF ROWS ROWS
INPUT NUMBER OF COLUMNS COLS
DECLARE A 2D INTEGER ARRAY MATRIX[ROWS][COLS]
INPUT MATRIX ELEMENTS INTO MATRIX
DECLARE INTEGER ARRAY ROWMIN[ROWS]
DECLARE INTEGER ARRAY COLMIN[COLS]
for I = 0 TO ROWS − 1 do
  SET MIN ← MATRIX[I][0]
  for J = 1 TO COLS − 1 do
    if MATRIX[I][J] < MIN then
      MIN ← MATRIX[I][J]
    end if
  end for
  ROWMIN[I] ← MIN
end for
for J = 0 TO COLS − 1 do
  SET MIN ← MATRIX[0][J]
  for I = 1 TO ROWS − 1 do
    if MATRIX[I][J] < MIN then
      MIN ← MATRIX[I][J]
    end if
  end for
  COLMIN[J] ← MIN
end for
PRINT MATRIX WITH EACH ROW FOLLOWED BY ITS ROW MINIMUM
PRINT COLUMN MINIMUMS IN A SINGLE LINE
STOP
```

## ❒ Code:-

```
    import java.util.Scanner;
public class MatrixMinValues {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        // Input number of rows and columns
        System.out.print("Enter number of rows: ");
        int rows = scanner.nextInt();

        System.out.print("Enter number of columns: ");
        int cols = scanner.nextInt();
```

```
int[][] matrix = new int[rows][cols];

// Input matrix elements
System.out.println("Enter matrix elements:");
for (int i = 0; i < rows; i++) {
    for (int j = 0; j < cols; j++) {
        matrix[i][j] = scanner.nextInt();
    }
}
int[] rowMin = new int[rows];
int[] colMin = new int[cols];

// Find row minimums
for (int i = 0; i < rows; i++) {
    int min = matrix[i][0];
    for (int j = 1; j < cols; j++) {
        if (matrix[i][j] < min) {
            min = matrix[i][j];
        }
    }
    rowMin[i] = min;
}
// Find column minimums
for (int j = 0; j < cols; j++) {
    int min = matrix[0][j];
    for (int i = 1; i < rows; i++) {
        if (matrix[i][j] < min) {
            min = matrix[i][j];
        }
    }
    colMin[j] = min;
}
// Print matrix with row minimums
System.out.println("\nMatrix with row minimums:");
for (int i = 0; i < rows; i++) {
    for (int j = 0; j < cols; j++) {
```

```
                System.out.print(matrix[i][j] + "\t");
            }
            System.out.println(rowMin[i]);
        }

        // Print column minimums
        System.out.println();
        for (int j = 0; j < cols; j++) {
            System.out.print(colMin[j] + "\t");
        }

        scanner.close();
    }
}
```

## ❒ Output:-

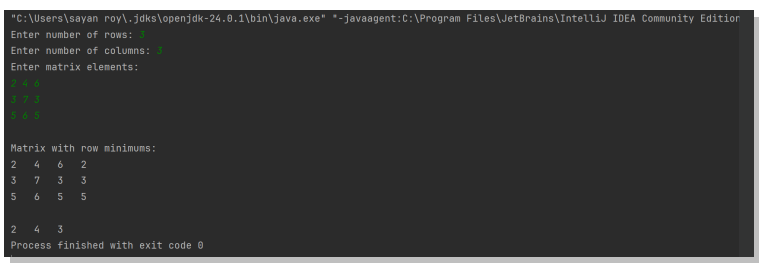



**Figure 2**

## ❒ Conclusion:-

The Java program efficiently reads a matrix from user input and computes the minimum values for each row and each column. It demonstrates the use of nested loops and conditional logic to traverse two-dimensional arrays. This assignment enhances understanding of matrix manipulation and reinforces key concepts such as array indexing, comparison operations, and formatted output in Java. It serves as a practical example of solving numerical problems using structured programming techniques.

## Problem Statement-3:-

**Write a program in java to delete all consonants from an input string and print the result string.**

### ❒ Objective:-

- **User Input:** To create a Java program that accepts a string from the user. The input may contain alphabets, numbers, symbols, and spaces.
- **Character Analysis:** To process each character in the string and identify whether it is a vowel, consonant, or non-letter. The identification is case-insensitive.
- **Vowel Retention:** To retain all vowels (A, E, I, O, U and their lowercase forms) in the output. These characters are preserved in their original order.
- **Consonant Removal:** To remove all consonant characters from the string. Only alphabetic consonants are excluded; other characters remain unchanged.
- **Result Construction:** To build the resulting string by appending valid characters (vowels and non-letters). A 'StringBuilder' or similar structure may be used for efficient concatenation.
- **Output Display:** To display the final string after removing consonants. The output reflects only vowels and non-alphabet characters from the original input.

## ❒ Algorithm:-

**Algorithm 3** Remove Consonants From a String

```
START
INPUT A STRING FROM THE USER AND STORE IT IN INPUT
INITIALIZE AN EMPTY STRING BUILDER RESULT
for EACH CHARACTER CH IN INPUT do
  if CH IS A VOWEL OR NOT A LETTER then
    APPEND CH TO RESULT
  end if
end for
DISPLAY RESULT AS THE STRING AFTER REMOVING CONSONANTS
STOP
```

## ❒ Code:-

```java
import java.util.Scanner;
public class RemoveConsonants {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        // Take input from user
        System.out.print("Enter a string: ");
        String input = scanner.nextLine();
        // Convert string to lowercase for uniformity
        StringBuilder result = new StringBuilder();
        for (char ch : input.toCharArray()) {
            if (isVowel(ch) || !Character.isLetter(ch)) {
                result.append(ch);
            }
        }
        // Print the result string
        System.out.println("String after removing
        consonants: " + result.toString());

        scanner.close();
    }
    // Function to check if a character is a vowel
    private static boolean isVowel(char ch) {
        return "AEIOUaeiou".indexOf(ch) != -1;
    }
```

```
}
```

## ❒ Output:-

```
"C:\Users\sayan roy\.jdks\openjdk-24.0.1\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Community Editior
Enter a string: Rabindranath Tagore was one of the most legendary Indian poets and novelists of all times
String after removing consonants: aiaa aoe a oe o e o eea Iia oe a oei o a ie

Process finished with exit code 0
```

**Figure 3**

## ❒ Conclusion:-

The Java program effectively removes all consonants from the input string while preserving vowels and non-letter characters. It utilizes character-level iteration, conditional checks, and string manipulation through the `StringBuilder` class. This assignment reinforces the understanding of string handling, method creation, and character classification in Java. It also highlights the importance of clean logic and efficient looping in solving text processing problems.

## Problem Statement-4:-

**A class called MyPoint, which models a 2D point with x and y coordinates. It contains:**

- **Two instance variables x (int) and y (int).**
- **A default (or "no-argument" or "no-arg") constructor that construct a point at the default location of (0, 0).**
- **A overloaded constructor that constructs a point with the given x and y coordinates.**
- **A method setXY() to set both x and y.**
- **A method getXY() which returns the x and y in a 2-element int array.**
- **A toString() method that returns a string description of the instance in the format "(x, y)".**
- **A method called distance(int x, int y) that returns the distance from this point to another point at the given (x, y) coordinates, Write the MyPoint class. Also write a test driver (called TestMyPoint) to test all the public methods defined in the class.**

### ❒ Objective:-

- **Class Definition:** Define a class named `MyPoint` to model a point in 2D space with integer coordinates `x` and `y`.
- **Default Constructor:** Create a no-argument constructor to initialize the point at (0, 0), representing the origin.

- **Parameterized Constructor:** Define an overloaded constructor that accepts two integers to set custom values of `x` and `y`.
- **Setting Values:** Implement a method `setXY()` that assigns values to both `x` and `y` simultaneously.
- **Returning Coordinates:** Implement `getXY()` to return a 2-element array containing the current values of `x` and `y`.
- **Custom String Output:** Override the `toString()` method to return a string in the format `"(x, y)"`.
- **Distance Calculation:** Write a method `distance(int x, int y)` that computes the distance from the current point to another point using the distance formula.
- **Test Class Creation:** Define a separate class `TestMyPoint` containing the `main()` method to test the functionality of the `MyPoint` class.
- **Object Instantiation and Testing:** In the test class, create one or more `MyPoint` objects and call all public methods to verify expected behavior.
- **Display Results:** Print the output from all test cases, including coordinates, string representation, and distance values.

## ❒ Algorithm:-

**Algorithm 4** Class `MyPoint` Functionality

**Class** MyPoint
Declare private integer variables `x`, `y`
**Constructor** MyPoint()
  Set `x = 0`, `y = 0`
**Constructor** MyPoint(x, y)
  Set `this.x = x`, `this.y = y`
**Method** setXY(x, y)
  Set `this.x = x`, `this.y = y`
**Method** getXY()
  Return array {`x, y`}
**Method** toString()
  Return string `"(x, y)"`
**Method** distance(x, y)
  Compute `dx = this.x - x`
  Compute `dy = this.y - y`
  Return `sqrt(dx * dx + dy * dy)`

**Algorithm 5** Testing Class `MyPoint` in `TestMyPoint`

Create object `p1` using default constructor
Print `p1` ⇒ (0, 0)
Create object `p2` using constructor with parameters (3, 4)
Print `p2` ⇒ (3, 4)
Call `p1.setXY(1, 2)`
Print updated `p1` ⇒ (1, 2)
Call `p1.getXY()` and store in `coords`
Print `coords[0], coords[1]` ⇒ (1, 2)
Call `p1.distance(3, 4)` and store in `dist`
Print `dist` ⇒ Expected output: 2.828...

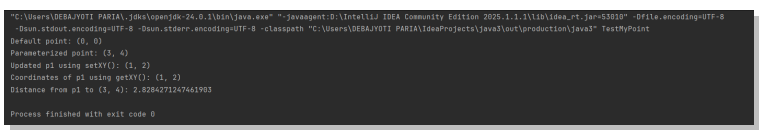
## ❒ Code:-

```
public class MyPoint {
// Instance variables
private int x;
private int y;
// Default constructor
public MyPoint() {
    this.x = 0;
    this.y = 0;
}
// Overloaded constructor
public MyPoint(int x, int y) {
```

```
        this.x = x;
        this.y = y;
    }
    // Setter method to set x and y
    public void setXY(int x, int y) {
        this.x = x;
        this.y = y;
    }
    // Getter method to return x and y in an array
    public int[] getXY() {
        return new int[]{x, y};
    }
    // toString method to return the point in (x, y) format
    @Override
    public String toString() {
        return "(" + x + ", " + y + ")";
    }
    // Method to calculate distance from this
    point to another (x, y)
    public double distance(int x, int y) {
        int dx = this.x - x;
        int dy = this.y - y;
        return Math.sqrt(dx * dx + dy * dy);
    }
}
public class TestMyPoint {
    public static void main(String[] args) {
        // Test default constructor
        MyPoint p1 = new MyPoint();
        System.out.println("Default point: " +
        p1); // (0, 0)
        // Test overloaded constructor
        MyPoint p2 = new MyPoint(3, 4);
        System.out.println("Parameterized point:
        " + p2); // (3, 4)
        // Test setXY()
```

```
        p1.setXY(1, 2);
        System.out.println("Updated p1 using
        setXY(): " + p1); // (1, 2)
        // Test getXY()
        int[] coords = p1.getXY();
        System.out.println("Coordinates of p1
        using getXY(): (" + coords[0] + ", " +
        coords[1] + ")");
        // Test distance()
        double dist = p1.distance(3, 4);
        System.out.println("Distance from p1 to
        (3, 4): " + dist); // Expected: 2.828...
    }
}
```

## ❒ Output:-



```
"C:\Users\DEBAJYOTI PARIA\.jdks\openjdk-24.0.1\bin\java.exe" "-javaagent:D:\IntelliJ IDEA Community Edition 2025.1.1.1\lib\idea_rt.jar=53010" -Dfile.encoding=UTF-8
 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath "C:\Users\DEBAJYOTI PARIA\IdeaProjects\java3\out\production\java3" TestMyPoint
Default point: (0, 0)
Parameterized point: (3, 4)
Updated p1 using setXY(): (1, 2)
Coordinates of p1 using getXY(): (1, 2)
Distance from p1 to (3, 4): 2.8284271247461903

Process finished with exit code 0
```

**Figure 4**

## ❒ Conclusion:-

The Java program successfully demonstrates the implementation of a class to represent a 2D point, including constructors, accessor and mutator methods, and a method to calculate the distance between points. Through the `MyPoint` and `TestMyPoint` classes, the assignment reinforces core object-oriented concepts such as encapsulation, method overloading, and the use of instance methods. It provides hands-on experience in class design and working with coordinate geometry in Java.

## Problem Statement-5:-

**Create a superclass 'Person' and two subclasses 'Student' and 'Staff'. The following are the instance variables and methods: a. For 'Person' instance variables: name:String, address:String. Initiate variable through constructor, incorporate one method setPerson() that updates Person variables , another method tostring() that shows Person details as "Person[name=?,address=?". b. For 'Student' sub class instance variables: program:String, year:String, fees:double. Initiate both 'Student' and 'Person' variables through constructor, incorporate one method setStudent() that updates both student and 'Person' data, another method tostring() that shows 'Person-Student' details as "Person[name=?,address=?,Program=?,Year=?,Fees=?". c. For 'Staff' subclass instance variables: school:String, pay:double. Initiate both 'Staff' and 'Person' variables through constructor, incorporate one method setStaff() that updates both 'staff' and 'Person' data, another method tostring() that shows 'Person-Staff' details as "Person[name=?,address=?,School=?,Pays=?". Write the classes and a test driver main class to test all functions mentioned above.**

### ❒ Objective:-

- **Define Superclass `Person`:** Create a superclass named `Person` with common attributes such as `name`, `address`, and

methods for setting and displaying information.

- **Create Subclass `Student`:** Extend the `Person` class to form a `Student` class with additional attributes like `program`, `year`, and `fee`.
- **Create Subclass `Staff`:** Similarly, create another subclass `Staff` with specific attributes such as `school` and `pay`.
- **Implement Constructors:** Define constructors in all classes for proper initialization of attributes, using `super` keyword where needed to invoke the superclass constructor.
- **Override `toString()` Methods:** Provide meaningful string representations for each class by overriding the `toString()` method to include both inherited and specific attributes.
- **Use Setter and Getter Methods:** Implement getter and setter methods to allow controlled access to private data fields and to demonstrate encapsulation.
- **Instantiate Classes in Main:** In a separate driver class, create objects of `Person`, `Student`, and `Staff`, initializing them with suitable values.
- **Demonstrate Polymorphism:** Use polymorphism by referring to subclass objects using superclass references and calling overridden methods.
- **Display Object Details:** Call the `toString()` methods for each object and print their information on the console for verification.
- **Test and Validate:** Verify the correctness of constructor chaining, method overriding, and data handling through sample output and debugging if needed.

## ❒ Algorithm:-

**Algorithm 6** Class `Person`, `Student`, and `Staff` Structure

```
Class Person
    Declare private variables: name, address
    Constructor(name, address): Initialize fields
    Method setPerson(name, address): Set new values
    Method toString(): Return "Person[name=..., address=...]"
Class Student extends Person
    Declare private variables: program, year, fees
    Constructor(name, address, program, year, fees):
        Call super(name, address), set remaining fields
    Method setStudent(...): Update inherited and own fields
    Override toString(): Return full student details
Class Staff extends Person
    Declare private variables: school, pay
    Constructor(name, address, school, pay):
        Call super(name, address), set remaining fields
    Method setStaff(...): Update inherited and own fields
    Override toString(): Return full staff details
```

**Algorithm 7** Test Program for `Person`, `Student`, and `Staff`

```
Create Person object p with name "John Doe", address "123 Main St"
Print p
Call p.setPerson("Jane Doe", "456 Oak Ave")
Print updated p
Create Student object s with ("Alice", "789 Park Blvd", "Computer Science", "2025", 45000.50)
Print s
Call s.setStudent("Alice Smith", "321 Elm St", "AI & ML", "2026", 50000.75)
Print updated s
Create Staff object st with ("Bob", "654 Maple Rd", "Greenwood High", 30000)
Print st
Call st.setStaff("Bob Johnson", "999 Pine St", "Sunshine School", 35000)
Print updated st
```

## ❒ Code:-

```java
public class Student extends Person {
private String program;
private String year;
private double fees;

public Student(String name, String address,
String program, String year, double fees) {
    super(name, address);
    this.program = program;
```

```
        this.year = year;
        this.fees = fees;
    }

    public void setStudent(String name, String
    address, String program, String year, double
    fees) {
        setPerson(name, address);
        this.program = program;
        this.year = year;
        this.fees = fees;
    }
    @Override
    public String toString() {
        return "Person[name=" + name + ",
        address=" + address + ", Program=" +
        program + ", Year=" + year + ", Fees=" +
        fees + "]";
    }
}
public class Staff extends Person {
    private String school;
    private double pay;

    public Staff(String name, String address,
    String school, double pay) {
        super(name, address);
        this.school = school;
        this.pay = pay;
    }
    public void setStaff(String name, String
    address, String school, double pay) {
        setPerson(name, address);
        this.school = school;
        this.pay = pay;
    }
```

```
    @Override
    public String toString() {
        return "Person[name=" + name + ",
        address=" + address + ", School=" +
        school + ", Pays=" + pay + "]";
    }
}
public class TestMain {
    public static void main(String[] args) {
        // Test Person

        Person p = new Person("John Doe", "123
        Main St");
        System.out.println(p);
        p.setPerson("Jane Doe", "456 Oak Ave");
        System.out.println("After update: " + p);

        // Test Student
        Student s = new Student("Alice", "789
        Park Blvd", "Computer Science", "2025",
        45000.50);
        System.out.println(s);
        s.setStudent("Alice Smith", "321 Elm St",
        "AI & ML", "2026", 50000.75);
        System.out.println("After update: " + s);

        // Test Staff

        Staff st = new Staff("Bob", "654 Maple
        Rd", "Greenwood High", 30000);
        System.out.println(st);
        st.setStaff("Bob Johnson", "999 Pine St",
        "Sunshine School", 35000);
        System.out.println("After update: " + st);
    }
}
```

## ❒ Output:-

```
"C:\Users\DEBAJYOTI PARIA\.jdks\openjdk-24.0.1\bin\java.exe" "-javaagent:D:\IntelliJ IDEA Community Edition 2025.1.1.1\lib\idea_rt.jar=53279" -Dfile.encoding=UTF-8
 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath "C:\Users\DEBAJYOTI PARIA\IdeaProjects\java3\out\production\java3" TestMain
Person[name=John Doe, address=123 Main St]
After update: Person[name=Jane Doe, address=456 Oak Ave]
Person[name=Alice, address=789 Park Blvd, Program=Computer Science, Year=2025, Fees=45000.5]
After update: Person[name=Alice Smith, address=321 Elm St, Program=AI & ML, Year=2026, Fees=50000.75]
Person[name=Bob, address=654 Maple Rd, School=Greenwood High, Pays=30000.0]
After update: Person[name=Bob Johnson, address=999 Pine St, School=Sunshine School, Pays=35000.0]

Process finished with exit code 0
```

**Figure 5**

## ❒ Conclusion:-

The Java program demonstrates the concept of inheritance by creating a superclass `Person` and extending it into subclasses `Student` and `Staff`. It effectively showcases constructor chaining using `super`, method overriding through the `toString()` method, and subclass-specific properties and methods. This assignment strengthens the understanding of object-oriented principles, particularly inheritance and polymorphism, enabling the design of reusable and organized class hierarchies in Java.

## Problem Statement-6:-

**Create a base class 'Square' having instance variable side:double. Initiate variable using constructor, a method 'getVolume() : double' that calculates volume and print it. Create a derived class 'Cylinder' having instance variable height:double. Initiate variables of both classes through constructor, override method 'getVolume() : double' to calculate volume of cylinder taking 'side' variable of base class as 'radius' and print it.**

### ❒ Objective:-

- **Understanding Inheritance:** To demonstrate how a derived class can extend the functionality of a base class by inheriting its properties.
- **Constructor Usage:** To learn how to initialize variables of both base and derived classes using constructors, including the use of the `super` keyword.
- **Method Overriding:** To practice method overriding by redefining the `getVolume()` method in the derived class to provide specific functionality.
- **Applying Mathematical Concepts:** To apply formulas for volume calculation of a cube and a cylinder using class methods.
- **Encapsulation and Reusability:** To reinforce the importance of encapsulating data and designing reusable components using object-oriented principles.
- **Polymorphism Implementation:** To understand how

runtime polymorphism works through method overriding in an inheritance hierarchy.

## ❒ Algorithm:-

**Algorithm 8** Class `Square` and `Cylinder` Definition

```
Class Square
  Declare protected double side
  Constructor(side): set this.side = side
  Method getVolume():
    Compute volume = side × side × side
    Print and return volume (treat as cube)
Class Cylinder extends Square
  Declare private double height
  Constructor(radius, height):
    Call super(radius)
    Set this.height = height
  Override Method getVolume():
    Compute volume = pi × side² × height
    Print and return volume (as cylinder)
```

**Algorithm 9** Testing Shapes in `TestShape`

```
Create Square object cube with side = 3
Call cube.getVolume() ⇒ Output: 27.0
Create Cylinder object cyl with radius = 3, height = 5
Call cyl.getVolume() ⇒ Output: π × 3² × 5 = 141.37...
```

## ❒ Code:-

```
public class Square {
protected double side;

public Square(double side) {
    this.side = side;
}

public double getVolume() {
    double volume = side * side * side;  //
    Treating it as cube volume
    System.out.println("Volume of Square (as
    Cube) = " + volume);
    return volume;
```

```
    }
}
public class Cylinder extends Square {
    private double height;

    public Cylinder(double radius, double height) {
        super(radius);  // 'side' of Square
        becomes 'radius' of Cylinder
        this.height = height;
    }
    @Override
    public double getVolume() {
        double volume = Math.PI * side * side *
        height;  // pi * r² * h
        System.out.println("Volume of Cylinder =
        " + volume);
        return volume;
    }
}
public class TestShape {
    public static void main(String[] args) {
        // Create Square object
        Square cube = new Square(3);
        cube.getVolume();  // 27.0

        // Create Cylinder object
        Cylinder cyl = new Cylinder(3, 5);
        cyl.getVolume();   // Should print
        Cylinder volume using radius = 3, height = 5
    }
}
```

## ❒ Output:-

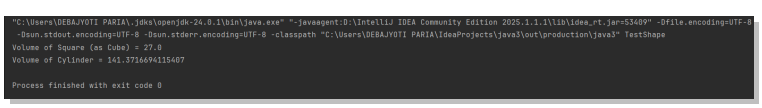



**Figure 6**

❒ **Conclusion:-**

The Java program demonstrates the use of inheritance and method overriding by modeling geometric shapes through the `Square` and `Cylinder` classes. The `Square` class is reused to represent a cube, while the `Cylinder` class extends it to calculate cylindrical volume using overridden behavior. This assignment highlights the importance of polymorphism, constructor chaining with `super`, and runtime method resolution. It reinforces how base classes can be extended and customized to solve more specific problems in object-oriented programming.

## Problem Statement-7:-

**Consider you are designing vehicles engine with 'speed:int, gear:int'. you can define your engine functionalities 'speedUp(value)' and 'changeGear(value) in an interface. The class which is implementing the interface should implement all the methods in the interface.**

### ❒ Objective:-

- **Understanding Interfaces:** To learn how to define and use interfaces in Java for specifying a contract of functionalities.
- **Abstraction in OOP:** To implement abstraction by separating engine behavior definitions from their implementation using interfaces.
- **Implementing Interface Methods:** To understand that any class implementing an interface must provide concrete implementations for all declared methods.
- **Encapsulation of Engine Properties:** To encapsulate engine-related data such as `speed` and `gear` within a class.
- **Customizing Behavior:** To allow specific behavior for speed and gear changes by implementing methods like `speedUp(value)` and `changeGear(value)`.
- **Designing Flexible Systems:** To design flexible and extensible systems where multiple classes can implement the same interface with different logic.

## ❒ Algorithm:-

**Algorithm 10** Interface `Engine` and Class `VehicleEngine`

**Interface** Engine
  Declare method `speedUp(value)`
  Declare method `changeGear(value)`
**Class** VehicleEngine **implements** Engine
  Declare private variables: `speed`, `gear`
  **Constructor**(speed, gear): Initialize values
  **Method** speedUp(value):
    Add `value` to `speed`
    Print updated speed
  **Method** changeGear(value):
    Set `gear = value`
    Print updated gear
  **Method** showStatus():
    Print current speed and gear

**Algorithm 11** Testing Engine Functionality in `TestVehicleEngine`

Create object `car` of class `VehicleEngine` with speed = 0, gear = 1
Call `car.showStatus()`
Call `car.speedUp(20)`
Call `car.changeGear(2)`
Call `car.speedUp(30)`
Call `car.changeGear(3)`
Call `car.showStatus()`

## ❒ Code:-

```
    // Interface defining engine functionalities
interface Engine {
    void speedUp(int value);
    void changeGear(int value);
}
// Class implementing the Engine interface
class VehicleEngine implements Engine {
    private int speed;
    private int gear;
    // Constructor
    public VehicleEngine(int speed, int gear) {
        this.speed = speed;
        this.gear = gear;
    }
```

```
    // Implement speedUp method
    @Override
    public void speedUp(int value) {
        speed += value;
        System.out.println("Speed increased to: "
        + speed + " km/h");
    }
    // Implement changeGear method
    @Override
    public void changeGear(int value) {
        gear = value;
        System.out.println("Gear changed to: " +
        gear);
    }
    // Display current status
    public void showStatus() {
        System.out.println("Current Speed: " +
        speed + " km/h, Gear: " + gear);
    }
}
// Test driver class
public class TestVehicleEngine {
    public static void main(String[] args) {
        VehicleEngine car = new VehicleEngine(0, 1);
        car.showStatus();
        car.speedUp(20);
        car.changeGear(2);
        car.speedUp(30);
        car.changeGear(3);
        car.showStatus();
    }
}
```

## ❒ Output:-

```
"C:\Users\sayan roy\.jdks\openjdk-24.0.1\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Community Edition
Current Speed: 0 km/h, Gear: 1
Speed increased to: 20 km/h
Gear changed to: 2
Speed increased to: 50 km/h
Gear changed to: 3
Current Speed: 50 km/h, Gear: 3

Process finished with exit code 0
```

**Figure 7**

## ❒ **Conclusion:-**

The Java program effectively demonstrates the concept of interfaces through the `Engine` interface and its implementation in the `VehicleEngine` class. It showcases how abstraction is achieved in Java by defining behavior contracts that must be implemented by concrete classes. The program also illustrates real-world functionality like speeding up and changing gears, helping to understand how interface-based design supports modular, flexible, and scalable software development.

## Problem Statement-8:-

**Write a program in java that handles both 'ArrayIndexOutOfBoundsException' and 'ArithmeticException'.**

### ❒ Objective:-

- **Understanding Exception Handling:**
  To learn how Java's exception mechanism works by using `try`, `catch`, and `finally` blocks. This helps manage runtime anomalies without abruptly terminating the program.
- **Handling Multiple Exceptions:**
  To demonstrate the handling of multiple types of exceptions (such as `ArrayIndexOutOfBoundsException`, using separate or multi-catch blocks in one program.
- **Improving Program Robustness:**
  To build fault-tolerant applications that anticipate and recover from common errors, resulting in more stable and user-friendly software.
- **Using Control Structures:**
  To integrate exception handling with conditional statements to define alternative execution paths and ensure uninterrupted program flow during error conditions.
- **Practicing Defensive Programming:**
  To follow best practices for error prevention and validation of user inputs, reducing the likelihood of unhandled exceptions and promoting reliable code execution.
- **Using Finally Block for Cleanup:**
  To ensure that important cleanup operations (like closing re-

sources) are executed regardless of whether an exception occurs or not.

– **Enhancing Code Debugging and Maintenance:** To use exception messages and stack traces to quickly locate the source of errors, simplifying the debugging and maintenance process of the application.

## ❒ Algorithm:-

**Algorithm 12** Exception Handling Demonstration

```
Declare integer array numbers = {10, 20, 30}
Declare integer result = 0
Access numbers[5] (Invalid index)
Assign to variable value
Divide value / 0 (division by zero)
Assign result to result
Print "Result:  " + result ArrayIndexOutOfBoundsException
Print "Error:  Tried to access an invalid index in the array." ArithmeticException
Print "Error:  Division by zero is not allowed."
Print "Program continues normally after exception handling."
```

## ❒ Code:-

```
public class ExceptionHandlingDemo {
public static void main(String[] args) {
    int[] numbers = {10, 20, 30};
    int result = 0;
    try {
        // This will cause ArrayIndexOutOfBoundsException
        int value = numbers[5];

        // This will cause ArithmeticException (division
        by zero)
        result = value / 0;

        System.out.println("Result: " + result);

    } catch (ArrayIndexOutOfBoundsException e) {
        System.out.println("Error: Tried to
```

```
            access an invalid index in the array.");
        } catch (ArithmeticException e) {
            System.out.println("Error: Division
            by zero is not allowed.");
        }

        System.out.println("Program continues
        normally after exception handling.");
    }
}
```

❒ **Output:-**

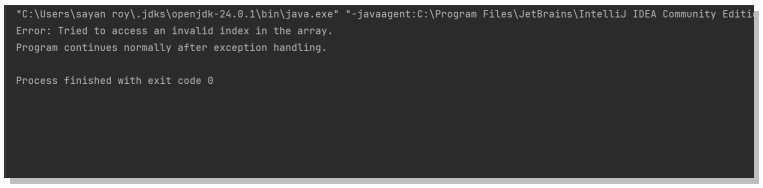



**Figure 8**

❒ **Conclusion:-**

The Java program clearly demonstrates the concept of exception handling using `try-catch` blocks. It handles both `ArrayIndexOutOfBoundsException` and `ArithmeticException`, showing how runtime errors can be gracefully managed without abruptly terminating the program. This assignment enhances understanding of robust programming practices, emphasizing the importance of error detection, exception handling, and ensuring program continuity in Java applications.

## Problem Statement-9:-

**Write a program in java that inputs students data(Regno, Sname, City, ContactNo) from user and insert into table Student info using JDBC connectivity. Also view all records in tabular format.**

### ❒ Objective:-

- **Understanding JDBC Connectivity:** To learn how to establish a connection between a Java application and a relational database using JDBC (Java Database Connectivity).
- **Data Insertion into Database:** To insert user input data (Registration Number, Student Name, City, and Contact Number) into a database table using SQL INSERT statements.
- **Retrieving and Displaying Data:** To fetch and display all records from the `Student_info` table in a structured, tabular format using SQL SELECT queries.
- **Prepared Statements and Input Handling:** To use `PreparedStatement` for secure and efficient execution of SQL commands while handling user input.
- **Database Design and Table Structure:** To design a basic database schema that models student data in a tabular form.
- **Developing Practical Database Applications:** To apply theoretical knowledge of databases in practical applications that involve both front-end input and back-end storage.

## ❒ Algorithm:-

**Algorithm 13** JDBC Student Database Insertion and Retrieval

Define database URL, username, and password
Initialize `Scanner sc` to read user input
Load MySQL JDBC Driver using `Class.forName()`
Establish connection to database using `DriverManager.getConnection()`
Prompt and read student details:
  Read `regno`, `sname`, `city`, `contactno`
Prepare SQL insert query with placeholders:
  `INSERT INTO Student_info (regno, sname, city, contactno) VALUES (?, ?, ?, ?)`
Set parameters in the `PreparedStatement`
Execute the insert using `pstmt.executeUpdate()`
Print "Student data inserted successfully!"
Create `Statement` object
Execute `SELECT * FROM Student_info`
Print table header for output
**while** ResultSet has more records **do**
  Print `regno, sname, city, contactno`
**end while**
Close `ResultSet, Statement, PreparedStatement, Connection`
ClassNotFoundException
Print "JDBC Driver not found."
SQLException
Print "Database error: " + error message
Close `Scanner`

## ❒ Code:-

```
import java.sql.*;
import java.util.Scanner;
public class StudentJDBC {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/your_database
        "; // change DB name
        String user = "your_username"; // your DB username
        String password = "your_password"; // your DB password
        Scanner sc = new Scanner(System.in);
        try {
            // Load JDBC Driver (optional for modern JDBC)
            Class.forName("com.mysql.cj.jdbc.Driver");
            // Establish connection
            Connection conn = DriverManager.getConnection(url,
            user, password);
            // Take input from user
```

```
System.out.print("Enter Reg No: ");
int regno = sc.nextInt();
sc.nextLine(); // consume newline
System.out.print("Enter Student Name: ");
String sname = sc.nextLine();
System.out.print("Enter City: ");
String city = sc.nextLine();
System.out.print("Enter Contact No: ");
String contactno = sc.nextLine();
// Insert into table
String insertSQL = "INSERT INTO Student_info (regno,
sname, city, contactno) VALUES (?, ?, ?, ?)";
PreparedStatement pstmt = conn.prepareStatement(
insertSQL);
pstmt.setInt(1, regno);
pstmt.setString(2, sname);
pstmt.setString(3, city);
pstmt.setString(4, contactno);
pstmt.executeUpdate();
System.out.println("\nStudent data
inserted successfully!\n");
// Display all records
Statement stmt = conn.createStatement();
ResultSet rs = stmt.executeQuery("SELECT * FROM
Student_info");
System.out.printf("%-10s %-20s %-15s %-15s\n",
"RegNo", "Name", "City", "ContactNo");
System.out.println("------------------------
-----------------");
while (rs.next()) {
    System.out.printf("%-10d %-20s %-15s %-15s\n",
            rs.getInt("regno"),
            rs.getString("sname"),
            rs.getString("city"),
            rs.getString("contactno"));
}
```

```
            // Close resources
            rs.close();
            stmt.close();
            pstmt.close();
            conn.close();
        } catch (ClassNotFoundException e) {
            System.out.println("JDBC Driver not found.");
        } catch (SQLException e) {
            System.out.println("Database error: " +
            e.getMessage());
        } finally {
            sc.close();
        }
    }
}
```

## ❒ Output:-

**Figure 9**

## ❒ Conclusion:-

The Java program successfully demonstrates the use of JDBC (Java Database Connectivity) to perform basic database operations such as inserting and retrieving student records from a MySQL database. It covers essential database interactions using `Connection`, `PreparedStatement`, `Statement`, and `ResultSet` classes. This assignment enhances understanding of database-driven application development and reinforces concepts of secure data handling, exception management, and SQL integration within Java programs.

## Problem Statement-10:-

**Write a Java program to remove a specific element from an array.**

### ❒ Objective:-

- **Understanding Array Operations:** To develop a clear understanding of how arrays work in Java, including traversal and manipulation.
- **Removing Elements Manually:** To implement logic for removing a specific element from an array, as Java arrays have fixed size and no built-in remove method.
- **Using Conditional Statements:** To practice decision-making using conditions for identifying the target element to remove.
- **Enhancing Logical Thinking:** To improve algorithmic thinking by handling array reconstruction after removing an element.
- **Working with User Input:** To read dynamic input from users for both the array and the element to be removed.

## ❒ Algorithm:-

**Algorithm 14** Remove First Occurrence of Element from Array

```
Prompt user to input array size n
Declare integer array arr[n]
Prompt user to input n elements into arr
Prompt user to input target element to remove
Declare newArr[n − 1]
Initialize found ← false, j ← 0
for i = 0 to n − 1 do
    if arr[i] = target and found = false then
        Set found ← true {Skip first match}
        continue
    end if
    if j < length of newArr then
        newArr[j] ← arr[i]
        j ← j + 1
    end if
end for
if found = false then
    Print "Element not found in the array."
else
    Print "Array after removing target:"
    for i = 0 to newArr.length − 1 do
        Print newArr[i]
    end for
end if
```

## ❒ Code:-

```java
import java.util.Scanner;
public class RemoveElementFromArray {

public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        // Input array size
        System.out.print("Enter the size of the array: ");
        int size = sc.nextInt();
        // Input array elements
        int[] arr = new int[size];
        System.out.println("Enter " + size + " elements:");
        for (int i = 0; i < size; i++) {
            arr[i] = sc.nextInt();
        }
```

```
        // Input element to remove
        System.out.print("Enter element to remove: ");
        int target = sc.nextInt();
        // Remove element
        int[] newArr = new int[size - 1];
        boolean found = false;
        int j = 0;
        for (int i = 0; i < size; i++) {
            if (arr[i] == target && !found) {
                found = true; // Skip the first occurrence
                continue;
            }
            if (j < newArr.length)
                newArr[j++] = arr[i];
        }

        // Display result
        if (!found) {
            System.out.println("Element not found in the
            array.");
        } else {
            System.out.println("Array after removing " + target
            + ":");
            for (int i = 0; i < newArr.length; i++) {
                System.out.print(newArr[i] + " ");
            }
            System.out.println();
        }

        sc.close();
    }
}
```

## ❒ Output:-

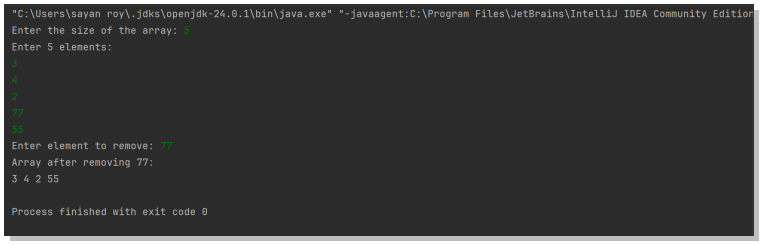



**Figure 10**

## ❒ Conclusion:-

The Java program effectively demonstrates how to remove a specified element from an array by creating a new array and omitting the target element. It utilizes conditional logic and loop control to handle array traversal and element shifting. This assignment reinforces the understanding of array manipulation, user input handling, and memory management through the use of dynamic array reconstruction. It also highlights the importance of validating user inputs and providing appropriate feedback.

## Problem Statement-11:-

**Write a Java program to insert an element (specific position) into an array.**

### ❒ Objective:-

- **Understanding Array Structure:** To gain a fundamental understanding of how arrays are organized and indexed in Java.
- **Element Insertion Logic:** To implement logic for inserting a new element into an existing array at a specific position by shifting elements.
- **Working with Fixed-Size Data Structures:** To handle fixed-size arrays in Java and create new arrays when modifications are required.
- **Input Validation and User Interaction:** To accept user input for the array, the element, and the target position, ensuring proper handling of input constraints.
- **Problem-Solving with Arrays:** To enhance problem-solving and algorithmic skills through manual manipulation of data structures.

## ❒ Algorithm:-

**Algorithm 15** Insert Element into an Array at Given Position

Prompt user to input array size $n$
Declare integer array $arr[n+1]$
Prompt user to input $n$ elements into $arr[0 \ldots n-1]$
Prompt user to input *element* and *position* to insert
**if** $position < 0$ or $position > n$ **then**
  Print "Invalid position! Must be between 0 and $n$"
**else**
  **for** $i = n$ to $position + 1$ step $-1$ **do**
    $arr[i] \leftarrow arr[i-1]$
  **end for**
  $arr[position] \leftarrow element$
  Print "Array after insertion:"
  **for** $i = 0$ to $n$ **do**
    Print $arr[i]$
  **end for**
**end if**

## ❒ Code:-

```
import java.util.Scanner;
public class InsertElementInArray {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        // Input array size
        System.out.print("Enter the size of the array: ");
        int size = sc.nextInt();
        // Create and input array
        int[] arr = new int[size + 1];  // +1 to
        accommodate new element
        System.out.println("Enter " + size + " elements:");
        for (int i = 0; i < size; i++) {
            arr[i] = sc.nextInt();
        }
        // Input element and position
        System.out.print("Enter the element to insert: ");
        int element = sc.nextInt();
        System.out.print("Enter the position (0 to " + size + "):
        ");
        int position = sc.nextInt();
```

```
        // Validate position
        if (position < 0 || position > size) {
            System.out.println("Invalid position! Must be between
            0 and " + size);
        } else {
            // Shift elements to the right
            for (int i = size; i > position; i--) {
                arr[i] = arr[i - 1];
            }
            // Insert the element
            arr[position] = element;

            // Output new array
            System.out.println("Array after insertion:");
            for (int i = 0; i <= size; i++) {
                System.out.print(arr[i] + " ");
            }
            System.out.println();
        }
        sc.close();
    }
}
```

## ❒ Output:-

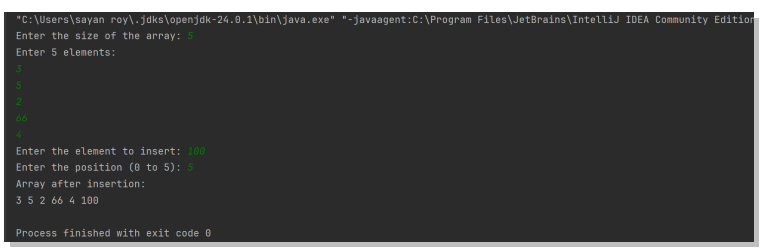



**Figure 11**

❒ **Conclusion:-**

The Java program successfully demonstrates how to insert an element at a specific position in an array. It includes input validation, element shifting, and efficient array traversal. This assignment helps in understanding core concepts of array manipulation, boundary checking, and memory management in Java. It reinforces logical thinking, precise index handling, and user-driven data insertion techniques in a structured programming approach.

## Problem Statement-12:-

**Write a Java program to find all pairs of elements in an array whose sum is equal to a specified number.**

### ❒ Objective:-

- **Understanding Array Traversal:** To practice iterating over arrays using nested loops for pairwise comparison of elements.
- **Solving a Real-World Problem:** To identify element pairs that satisfy a given condition (sum = target), a common problem in algorithm and interview practice.
- **Improving Logical Thinking:** To enhance problem-solving and algorithmic thinking by implementing efficient search strategies.
- **Working with Conditional Statements:** To use conditional logic to check for pairs that satisfy a mathematical condition.
- **User Interaction and Input Handling:** To accept dynamic user input for the array elements and target sum, ensuring real-time processing of data.

## ❒ Algorithm:-

**Algorithm 16** Find Pairs in Array with Given Sum

Prompt user to enter array size $n$
Declare integer array $arr[n]$
Prompt user to enter $n$ elements into $arr[0 \ldots n-1]$
Prompt user to enter target sum $T$
Set flag $found \leftarrow false$
**for** $i \leftarrow 0$ to $n-2$ **do**
  **for** $j \leftarrow i+1$ to $n-1$ **do**
    **if** $arr[i] + arr[j] = T$ **then**
      Print "$arr[i]$ + $arr[j]$ = $T$"
      $found \leftarrow true$
    **end if**
  **end for**
**end for**
**if** not $found$ **then**
  Print "No pairs found"
**end if**

## ❒ Code:-

```
import java.util.Scanner;
public class FindPairsWithSum {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        // Input array size and elements
        System.out.print("Enter the size of the array: ");
        int size = sc.nextInt();
        int[] arr = new int[size];
        System.out.println("Enter " + size + " array elements:");
        for (int i = 0; i < size; i++) {
            arr[i] = sc.nextInt();
        }
        // Input target sum
        System.out.print("Enter the target sum: ");
        int target = sc.nextInt();
        boolean found = false;
        System.out.println("Pairs with sum " + target + " are:");
        // Find all pairs
        for (int i = 0; i < size - 1; i++) {
            for (int j = i + 1; j < size; j++) {
```

```
                if (arr[i] + arr[j] == target) {
                    System.out.println(arr[i] + " + "
                    + arr[j] + " = " + target);
                    found = true;
                }
            }
        }
        if (!found) {
            System.out.println("No pairs found.");
        }
        sc.close();
    }
}
```

## ❒ Output:-

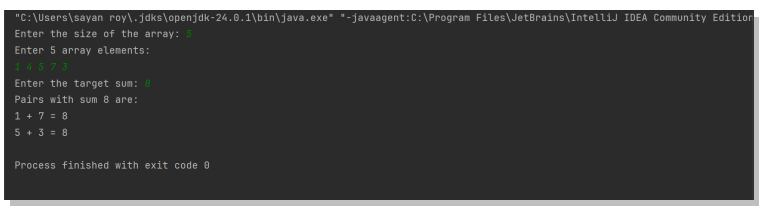



**Figure 12**

## ❒ Conclusion:-

The Java program efficiently identifies all pairs of elements in an array whose sum matches a user-defined target value. It demonstrates the use of nested loops, conditional checks, and boolean flags to handle pairwise comparison. This assignment strengthens the understanding of array traversal, combination logic, and user input handling.

## Problem Statement-13:-

**Write a Java program to remove the duplicate elements of a given array and return the new length of that array.**

### ❒ Objective:-

- **Understanding Array Manipulation:** To learn how to manipulate array data by identifying and eliminating duplicate values.
- **Practicing Sorting Techniques:** To use array sorting as a strategy to simplify the detection and removal of duplicates.
- **Improving Algorithmic Thinking:** To implement logic that filters unique values and constructs a new array of non-repeating elements.
- **Calculating Effective Array Size:** To determine and return the updated length of the array after removing duplicates.
- **Handling User Input:** To accept dynamic input from users and process it effectively through iterative and conditional logic.

## ❒ Algorithm:-

**Algorithm 17** Remove Duplicates from Array

Prompt user to enter array size $n$
Declare integer array $arr[0 \ldots n-1]$
Prompt user to input $n$ integers into $arr$
Initialize empty LinkedHashSet $uniqueSet$
**for** each element $num$ in $arr$ **do**
  Add $num$ to $uniqueSet$ (automatically ignores duplicates)
**end for**
Create array $uniqueArray$ of size $|uniqueSet|$
Set index $i \leftarrow 0$
**for** each $num$ in $uniqueSet$ **do**
  $uniqueArray[i] \leftarrow num$
  $i \leftarrow i + 1$
**end for**
Print "Array after removing duplicates:"
Print all elements of $uniqueArray$
Print "New length of array: $|uniqueArray|$"

## ❒ Code:-

```
import java.util.Scanner;
import java.util.LinkedHashSet;
public class RemoveDuplicatesFromArray {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        // Input array size
        System.out.print("Enter the size of the array: ");
        int size = sc.nextInt();
        int[] arr = new int[size];
        System.out.println("Enter " + size + " elements:");
        for (int i = 0; i < size; i++) {
        arr[i] = sc.nextInt();}
        LinkedHashSet<Integer> uniqueSet = new LinkedHashSet<>();
        for (int num : arr) {
        uniqueSet.add(num); }
        // Convert back to array (if needed) and display result
        int[] uniqueArray = new int[uniqueSet.size()];
        int i = 0;
        for (int num : uniqueSet) {
            uniqueArray[i++] = num;
```

```
        }
        // Output results
        System.out.println("Array after removing duplicates:");

        for (int val:uniqueArray)
        {
            System.out.print(val + " ");
        }
    System.out.println("Newlengt array:"+uniqueArray.length);

        sc.close();
    }
}
```

## ❒ Output:-

**Figure 13**

## ❒ Conclusion:-

The Java program effectively demonstrates how to remove duplicate elements from an array using the `LinkedHashSet` class, which maintains insertion order while eliminating repeated values. This approach simplifies the logic for handling duplicates and ensures clean, concise output. The assignment enhances the understanding of Java Collections, set operations, and array-to-collection conversions. It also reinforces concepts of efficient data handling and memory management in real-world programming tasks.

## Problem Statement-14:-

**Write a Java program to find the length of the longest consecutive elements sequence from a given unsorted array of integers. Sample array: [49, 1, 3, 200, 2, 4, 70, 5] The longest consecutive elements sequence is [1, 2, 3, 4, 5], therefore the program will return its length 5.**

### ❒ Objective:-

- **Understanding Sequence Detection:** To learn how to identify consecutive sequences within an unsorted array of integers.
- **Working with Unordered Data:** To handle and process unsorted data using data structures such as arrays and sets.
- **Applying Efficient Algorithms:** To implement an optimized algorithm (using HashSet or sorting) to find the longest sequence without unnecessary computation.
- **Enhancing Problem-Solving Skills:** To develop algorithmic thinking by solving a classic array-based problem commonly asked in coding interviews.
- **Practicing Java Collections:** To apply Java collections like `HashSet` for fast lookups and sequence tracking.
- **Handling Edge Cases:** To account for duplicate elements and disconnected number sequences during processing.

## ❒ Algorithm:-

**Algorithm 18** Find Length of Longest Consecutive Sequence

```
Given array arr of integers
Call function findLongestConsecutiveSequence(arr)
findLongestConsecutiveSequence(nums)
if nums.length == 0 then
    return 0
end if
Initialize empty HashSet numSet
for each number num in nums do
    Add num to numSet
end for
Initialize longestStreak ← 0
for each num in numSet do
    if numSet does not contain num − 1 then
        currentNum ← num
        currentStreak ← 1
        while numSet contains currentNum + 1 do
            currentNum ← currentNum + 1
            currentStreak ← currentStreak + 1
        end while
        longestStreak ← max(longestStreak, currentStreak)
    end if
end for
return longestStreak
```

## ❒ Code:-

```
import java.util.HashSet;
public class LongestConsecutiveSequence {
    public static void main(String[] args) {
        int[] arr = {49, 1, 3, 200, 2, 4, 70, 5};

System.out.println("Length oflongestconsecutivesequence: "
+ findLongestConsecutiveSequence(arr));
    }
    public static int findLongestConsecutiveSequence(int[] nums)
    {
        if (nums.length == 0) return 0;

        HashSet<Integer> numSet = new HashSet<>();
        for (int num : nums) {
            numSet.add(num);
```

```
        }
        int longestStreak = 0;
        for (int num : numSet) {
            if (!numSet.contains(num - 1)) {
                int currentNum = num;
                int currentStreak = 1;
                while (numSet.contains(currentNum + 1)) {
                    currentNum++;
                    currentStreak++;
                }
    longestStreak = Math.max(longestStreak, currentStreak);
            }
        }
        return longestStreak;
    }
}
```

## ❒ Output:-

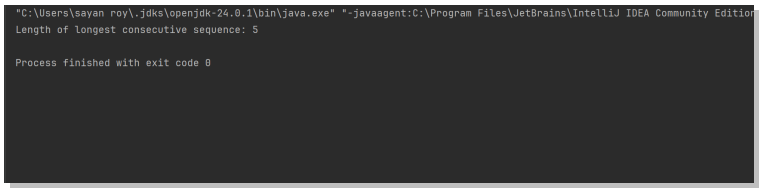



**Figure 14**

## ❒ Conclusion:-

The Java program efficiently calculates the length of the longest consecutive sequence within an unsorted array using a `HashSet` for constant-time lookups. By checking only for the start of sequences and extending them conditionally, the solution avoids unnecessary comparisons and improves performance. This assignment strengthens the understanding of set-based algorithms, loop optimization, and problem-solving using hashing techniques in Java.

## Problem Statement-15:-

**Write a java program to compare two strings lexicographically.**

### ❒ Objective:-

- **Understanding String Comparison:** To learn how to compare two strings lexicographically using built-in Java methods.
- **Exploring the `compareTo()` Method:** To demonstrate the use of the `String.compareTo()` method for determining the ordering of strings based on Unicode values.
- **Handling Alphabetical Ordering:** To understand how strings are ordered in dictionaries and how Java handles such ordering internally.
- **Practicing Conditional Statements:** To implement logic that interprets the result of a lexicographical comparison and outputs meaningful messages.
- **Strengthening String Manipulation Skills:** To build proficiency in working with Java's `String` class and related methods.

## ❒ Algorithm:-

**Algorithm 19** Compare Two Strings Lexicographically

```
Input: Two strings str1 and str2
Read str1 from user
Read str2 from user
Set result ← str1.compareTo(str2)
if result == 0 then
    Print "Both strings are equal."
else if result ¡ 0 then
    Print str1 is lexicographically less than str2
else
    Print str1 is lexicographically greater than str2
end if
```

## ❒ Code:-

```
import java.util.Scanner;
public class CompareStringsLexicographically {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        // Input two strings
        System.out.print("Enter first string: ");
        String str1 = sc.nextLine();
        System.out.print("Enter second string: ");
        String str2 = sc.nextLine();
        // Lexicographic comparison
        int result = str1.compareTo(str2);
        // Output result
        if (result == 0) {
            System.out.println("Both strings are equal.");
        } else if (result < 0) {
    System.out.println("\"" + str1 + "\" is lexicographically
 less than \"" + str2 + "\"");
        } else {
    System.out.println("\"" + str1 + "\" is lexicographically
    greater than \"" + str2 + "\"");
        }
        sc.close();
    }
}
```

## ❒ Output:-

**Figure 15**

❒ **Conclusion:-** This Java program demonstrates lexicographical comparison of two strings using the `compareTo()` method. It takes two strings as input and compares them based on Unicode values of characters. The result determines whether the first string is equal to, less than, or greater than the second string. This reinforces concepts of string manipulation, conditional branching, and understanding lexicographic order in Java.

## Problem Statement-16:-

**Write a Java program to find whether a region in the current string matches a region in another string. Sample Output: str1[0 - 7] == str2[28 - 35]? true str1[9 - 15] == str2[9 - 15]? false**

## ❒ Objective:-

- **Understanding String Comparison Techniques:** To learn how to compare specific parts (substrings) of two strings rather than comparing entire strings.
- **Using `regionMatches()` Method:** To explore the usage of the `regionMatches()` method in the `String` class for partial string comparison.
- **Improving String Manipulation Skills:** To gain hands-on experience in working with string indexing and substring matching.
- **Practicing Conditional Logic:** To apply conditional checks based on the outcome of region comparison for logical decision making.
- **Enhancing Debugging and Output Interpretation:** To interpret substring match results and print them clearly in a formatted manner.

## ❒ Algorithm:-

**Algorithm 20** Compare Substrings Using `regionMatches()`

```
str1 ← "Welcome to Java Programming"
str2 ← "Let's start Java Programming now"
match1 ← str1.regionMatches(0, str2, 28, 8)
match2 ← str1.regionMatches(9, str2, 9, 7)
Print "str1[0 - 7] == str2[28 - 35]? " + match1
Print "str1[9 - 15] == str2[9 - 15]? " + match2
```

## ❒ Code:-

```
public class RegionMatchExample {
public static void main(String[] args) {
        String str1 = "Welcome to Java Programming";
        String str2 = "Let's start Java Programming now";

        // Compare substrings using regionMatches()
        boolean match1 = str1.regionMatches(0, str2, 28, 8);
        boolean match2 = str1.regionMatches(9, str2, 9, 7);

        // Output results

System.out.println("str1[0 - 7] == str2[28 - 35]? " + match1);
System.out.println("str1[9 - 15] == str2[9 - 15]? " + match2);
    }
}
```

## ❒ Output:-

**Figure 16**

## ❒ Conclusion:-

The Java program demonstrates the use of the `regionMatches()` method to compare specific regions (substrings) of two strings. It provides insight into how substring-level comparisons can be efficiently performed without extracting substrings manually. This assignment enhances the understanding of Java's built-in string manipulation methods, and their role in implementing precise and performance-optimized text comparison operations.

## Problem Statement-17:-

**Write a Java program to print all permutations of a given string with repetition. Sample Output: The given string is: PQR The permuted strings are: PPP, PPQ, PPR, RRP, RRQ, RRR**

### ❒ Objective:-

- **Understanding String Permutations with Repetition:** To learn how to generate all possible string permutations when characters are allowed to repeat.
- **Implementing Nested Loops for Permutation Logic:** To apply nested loops effectively for generating combinations of characters from the input string.
- **Practicing Character Access and Concatenation:** To improve skills in accessing individual characters and forming new string patterns.
- **Improving Loop Control and Iteration:** To gain hands-on experience in iterating through multiple levels to generate repeated patterns.
- **Enhancing Output Formatting Skills:** To display generated permutations in a clear, structured, and readable format.

## ❒ Algorithm:-

**Algorithm 21** Generate Permutations with Repetition

```
Function main()
    input ← "PQR"
    length ← length of input
    Print "The given string is: " + input
    Print "The permuted strings are:"
    generatePermutations(input, "", length)

Function generatePermutations(input, result, length)
if length of result == length then
    Print result
    return
end if
for each character in input do
    generatePermutations(input, result + character, length)
end for
```

## ❒ Code:-

```
public class PermutationsWithRepetition {
public static void main(String[] args) {
        String input = "PQR";
        int length = input.length();
        System.out.println("The given string is: " + input);
        System.out.println("The permuted strings are:");
        generatePermutations(input, "", length);
    }

public static void generatePermutations
(String input, String result, int length) {
        if (result.length() == length) {
            System.out.println(result);
            return;
        }
        for (int i = 0; i < input.length(); i++) {

generatePermutations(input, result + input.charAt(i), length);

        }
    }
```

```
}
```

## ❒ Output:-

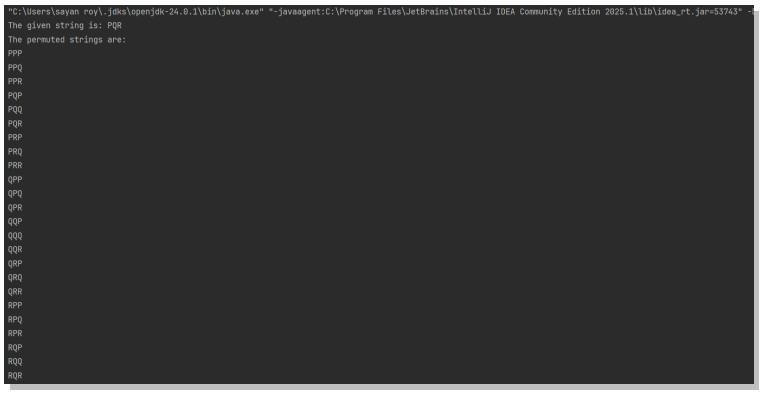



**Figure 17**

## ❒ Conclusion:-

The Java program successfully generates all possible permutations with repetition for a given string using recursion. It demonstrates the use of backtracking and string concatenation within a recursive method to build each permutation. This assignment enhances the understanding of recursive problem-solving techniques, especially in combinatorics, and provides a foundation for exploring more complex algorithms involving string manipulation and enumeration.

## Problem Statement-18:-

**Write a Java method to count all words in a string. Test Data: Input the string: The quick brown fox jumps over the lazy dog. Expected Output: Number of words in the string: 9**

### ❒ Objective:-

- **Understanding String Tokenization:** To learn how to break a string into individual words using built-in methods like `split()`.
- **Handling Whitespace and Input Variations:** To correctly manage extra spaces, tabs, and newline characters when counting words.
- **Practicing Conditional and Null Checks:** To implement safe checks for null or empty strings to avoid runtime errors.
- **Improving User Interaction via Console Input:** To use `Scanner` for accepting user input and processing it efficiently.
- **Enhancing Output Formatting and Validation:** To display the total word count in a user-friendly and formatted manner.

## ❒ Algorithm:-

**Algorithm 22** Count Number of Words in a String

```
Function main()
    Read input from user
    wordCount ← countWords(input)
    Print "Number of words in the string: " + wordCount

Function countWords(str)
if str is null OR trimmed str is empty then
   return 0
end if
words[] ← split str.trim() by one or more spaces
return length of words[]
```

## ❒ Code:-

```
import java.util.Scanner;
public class CountWordsInString {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        // Input the string
        System.out.print("Input the string: ");
        String input = sc.nextLine();

        // Call method to count words
        int wordCount = countWords(input);

        // Output the result
 System.out.println("Number of words in the string: "+wordCount);
        sc.close();
    }
    // Method to count words
    public static int countWords(String str) {
// Trim leading/trailing spaces and split by one or more spaces
        if (str == null || str.trim().isEmpty()) {
            return 0;
        }
        String[] words = str.trim().split("\\s+");
        return words.length;
```

```
        }
    }
```

## ❒ Output:-

```
"C:\Users\DEBAJYOTI PARIA\.jdks\openjdk-24.0.1\bin\java.exe" "-javaagent:D:\IntelliJ IDEA Community Edition 2025.1.1.1\lib\idea_rt.jar=53880" -Dfile.encoding=UTF-8 -Dsun.stdout
 .encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath "C:\Users\DEBAJYOTI PARIA\IdeaProjects\java4\out\production\java4" CountWordsInString
Input the string: Debajyoti is a good boy
Number of words in the string: 5

Process finished with exit code 0
```

**Figure 18**

## ❒ Conclusion:-

The Java program accurately counts the number of words in a given string by using string trimming and regular expression-based splitting. It handles various spacing scenarios, including multiple spaces and leading/trailing whitespace. This assignment strengthens string manipulation skills and introduces basic text processing techniques in Java, which are essential in many real-world applications such as form validation, parsing, and natural language processing.

## Problem Statement-19:-

**Write a program in java to create Box class with parameterized constructor with an object argument to initialize length, breadth and height also create a function volume which returns the volume of the box and print it in main method.**

### ❒ Objective:-

- **Understanding Object as Constructor Argument:** To learn how to use an object as a parameter in a constructor for initializing another object's data members.
- **Applying Class and Object Concepts:** To reinforce knowledge of class creation, object instantiation, and constructor overloading.
- **Practicing Data Encapsulation:** To encapsulate box dimensions and provide a method to compute volume based on internal data.
- **Enhancing Method Creation Skills:** To define and invoke a method that performs a computation (volume) and returns the result.
- **Improving Output Presentation:** To display the computed volume in a clear and formatted way from the `main()` method.

## ❒ Algorithm:-

**Algorithm 23** Box Volume Using Constructor Overloading

```
Class Box
  Private Variables: length, breadth, height
  Constructor Box(Box other)
    length ← other.length
    breadth ← other.breadth
    height ← other.height
  Constructor Box(length, breadth, height)
    this.length ← length
    this.breadth ← breadth
    this.height ← height
  Method volume()
    return length × breadth × height

Function main()
  originalBox ← new Box(3.0, 4.0, 5.0)
  copiedBox ← new Box(originalBox)
  Print "Volume of original box: " + originalBox.volume()
  Print "Volume of copied box: " + copiedBox.volume()
```

## ❒ Code:-

```
class Box {
private double length;
private double breadth;
private double height;
// Parameterized constructor with object argument
    public Box(Box other) {
        this.length = other.length;
        this.breadth = other.breadth;
        this.height = other.height;
    }
// Parameterized constructor to create initial box
    public Box(double length, double breadth, double height) {
        this.length = length;
        this.breadth = breadth;
        this.height = height;
    }
// Method to calculate volume
    public double volume() {
        return length * breadth * height;
```

```
    }
}
public class BoxDemo {
    public static void main(String[] args) {
        // Create original box
        Box originalBox = new Box(3.0, 4.0, 5.0);

        // Create another box using object of the first
        Box copiedBox = new Box(originalBox);

        // Print volumes

System.out.println("Volumeoforiginal box:"+originalBox.volume());
System.out.println("Volume of copied box: "+ copiedBox.volume());
    }
}
```

## ❒ Output:-

```
"C:\Users\DEBAJYOTI PARIA\.jdks\openjdk-24.0.1\bin\java.exe" "-javaagent:D:\IntelliJ IDEA Community Edition 2025.1.1.1\lib\idea_rt.jar=53924" -Dfile.encoding=UTF-8 -Dsun.stdout
 .encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath "C:\Users\DEBAJYOTI PARIA\IdeaProjects\java4\out\production\java4" BoxDemo
Volume of original box: 60.0
Volume of copied box: 60.0

Process finished with exit code 0
```

**Figure 19**

## ❒ Conclusion:-

The Java program illustrates the concept of a copy constructor by initializing one object using another of the same class. It reinforces object-oriented programming principles such as encapsulation and constructor overloading. By copying the properties of an existing Box object, this program demonstrates how to create deep copies of objects, which is a crucial technique in avoiding unintentional reference sharing and maintaining data integrity in real-world applications.

## Problem Statement-20:-

**Write a program in java with class Employee and do the following operations on it**

**Create two constructor default and with Object as parameter to initialize class variables.**

**Create a function Calculate which calculates the pf and allowances on the salary of employee and return the all values as an object.**

### ❒ Objective:-

- **Understanding Constructor Overloading:** To learn how to define and use both default and parameterized constructors, including object copy constructors.
- **Applying Object-Oriented Principles:** To reinforce core OOP concepts like encapsulation, object passing, and method return types.
- **Practicing Salary Component Calculations:** To implement a method that calculates PF (Provident Fund), allowances, and other salary components.
- **Returning Multiple Values via Objects:** To return multiple calculated values using an object instead of primitive return types.
- **Enhancing Output Formatting and Data Handling:** To display employee details and salary breakdown in a well-structured and readable format.

## ❒ Algorithm:-

**Algorithm 24** Employee Salary Calculation with Copy Constructor

```
Class Employee
  Variables: name, salary, pf, allowance, totalSalary
  Constructor Employee()
    name ← "Unknown"
    salary, pf, allowance, totalSalary ← 0
  Constructor Employee(Employee e)
    Copy all fields from e to this
  Constructor Employee(name, salary)
    this.name ← name
    this.salary ← salary
  Method calculate()
    result ← new Employee(this)
    result.pf ← 0.12 × salary
    result.allowance ← 0.20 × salary
    result.totalSalary ← salary + allowance - pf
    return result
  Method display()
    Print all employee details

Function main()
  e1 ← new Employee("John Doe", 50000)
  result ← e1.calculate()
  result.display()
```

## ❒ Code:-

```
class Employee {
String name;
double salary;
double pf;
double allowance;
double totalSalary;
// Default constructor
public Employee() {
        name = "Unknown";
        salary = 0;
        pf = 0;
        allowance = 0;
        totalSalary = 0;
}
// Constructor with Employee object as parameter.
```

```
public Employee(Employee e) {
        this.name = e.name;
        this.salary = e.salary;
        this.pf = e.pf;
        this.allowance = e.allowance;
        this.totalSalary = e.totalSalary;
}

public Employee(String name, double salary) {
        this.name = name;
        this.salary = salary;
}
// Method to calculate PF, Allowance,and Total Salary
public Employee calculate() {
Employee result = new Employee(this);
        result.pf = 0.12 * salary;          // 12% PF
        result.allowance = 0.20 * salary;   // 20% Allowance
     result.totalSalary = salary + result.allowance - result.pf;
        return result;
}
    // Display employee details
public void display() {
        System.out.println("Employee Name     : " + name);
        System.out.println("Basic Salary      : " + salary);
        System.out.println("PF (12%)          : " + pf);
        System.out.println("Allowance (20%)   : " + allowance);
        System.out.println("Total Salary      : " + totalSalary);
        System.out.println("----------------------------------");
    }
}
// Test Driver
public class EmployeeDemo {
    public static void main(String[] args) {
        // Create employee object with basic details
        Employee e1 = new Employee("John Doe", 50000);
```

```
        // Calculate PF, Allowance, and Total Salary
        Employee result = e1.calculate();

        // Display result
        result.display();
    }
}
```

## ❒ Output:-

```
"C:\Users\DEBAJYOTI PARIA\.jdks\openjdk-24.0.1\bin\java.exe" "-javaagent:D:\IntelliJ IDEA Community Edition 2025.1.1.1\lib\idea_rt.jar=51677" -Dfile.encoding=UTF-8
  -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath "C:\Users\DEBAJYOTI PARIA\IdeaProjects\java3\out\production\java3" EmployeeDemo
Employee Name     : John Doe
Basic Salary      : 50000.0
PF (12%)          : 6000.0
Allowance (20%)   : 10000.0
Total Salary      : 54000.0
-----------------------------------

Process finished with exit code 0
```

**Figure 20**

## ❒ Conclusion:-

This Java program effectively demonstrates constructor overloading, including the use of a copy constructor, to manage employee salary-related data. It calculates Provident Fund (PF), Allowance, and Total Salary based on a given basic salary and encapsulates this logic within a method. By separating initialization, computation, and display, the program maintains clean structure and readability, reflecting good object-oriented design practices.

## Problem Statement-21:-

**Write a program to create your own exception as NegativeSizeException whenever negative values are put in an array.**

### ❒ Objective:-

- **Understanding User-Defined Exceptions:** To learn how to create custom exception classes in Java by extending the `Exception` class.
- **Applying Exception Handling Mechanism:** To implement `try-catch` blocks to detect and handle exceptional cases such as negative array sizes.
- **Improving Input Validation:** To validate user inputs and throw exceptions proactively when inputs do not meet defined constraints.
- **Practicing Constructor Overriding in Exception Class:** To define custom constructors and messages within user-defined exception classes.
- **Enhancing Robustness and User Feedback:** To ensure the program fails gracefully with meaningful error messages instead of crashing on invalid input.

## ❒ Algorithm:-

**Algorithm 25** Custom Exception for Negative Array Size

```
Class NegativeSizeException extends Exception
  Constructor NegativeSizeException(message)
    Call superclass constructor with message

Function main()
  size ← -5 {Change this to test}
  try
    if size < 0 then
      throw NegativeSizeException("Array size cannot be negative:  " + size)
    arr ← new array of size size
    Print "Array of size size created successfully."
  catch NegativeSizeException e
    Print "Caught Exception: " + e.getMessage()
```

## ❒ Code:-

```
    // Custom exception class
class NegativeSizeException extends Exception {
    public NegativeSizeException(String message) {
        super(message);
    }
}
public class NegativeSizeDemo {
    public static void main(String[] args) {
        try {
            int size = -5;
            // Check for negative array size
    if (size < 0) {
 throw new NegativeSizeException("Array size cannot be negative:
 " + size);
            }
int[] arr = new int[size];  // Only created if size is valid
            System.out.println("Array of size " + size +
            " created successfully.");
        } catch (NegativeSizeException e) {
     System.out.println("Caught Exception: "+ e.getMessage());

        }
```

```
        }
    }
```

❒ **Output:-**

```
"C:\Users\DEBAJYOTI PARIA\.jdks\openjdk-24.0.1\bin\java.exe" "-javaagent:D:\IntelliJ IDEA Community Edition 2025.1.1.1\lib\idea_rt.jar=51756" -Dfile.encoding=UTF-8
 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath "C:\Users\DEBAJYOTI PARIA\IdeaProjects\java3\out\production\java3" NegativeSizeDemo
Caught Exception: Array size cannot be negative: -5

Process finished with exit code 0
```

**Figure 21**

❒ **Conclusion:-** This Java program demonstrates the use of custom exception handling by creating a user-defined exception class `NegativeSizeException`. It checks whether an array size is negative before creation and throws the custom exception if so. This approach shows how to enhance error reporting and program robustness by handling specific application-level errors more meaningfully.

## Problem Statement-22:-

**Create a class Student with following operations**

**create parameterized constructor to initialize the objects.**

**create a function isEqual() to check whether the two objects are equal or not which returns the Boolean value and gets two objects.**

**print the result in main method if objects are equals or not (take variables as your assumption)**

### ❒ Objective:-

- **Understanding Object Comparison:** To implement a method that compares two objects' data members to determine logical equality.
- **Practicing Constructor Initialization:** To learn how to use parameterized constructors to initialize object attributes effectively.
- **Applying Boolean Logic and Return Types:** To return a boolean value from a user-defined method based on conditional checks.
- **Reinforcing Object-Oriented Principles:** To develop a clear understanding of object creation, passing objects as parameters, and encapsulating logic.
- **Displaying Logical Results Clearly:** To display meaningful output in the `main()` method indicating whether two objects are equal or not.

## ❒ Algorithm:-

**Algorithm 26** Check Equality of Two Student Objects

```
Class Student
  rollNo:  int
  name:  String

Constructor Student(rollNo, name)
  this.rollNo ← rollNo
  this.name ← name

Static Method isEqual(s1, s2)
  return (s1.rollNo == s2.rollNo) AND (s1.name equals s2.name)

Function main()
  student1 ← new Student(101, "Alice")
  student2 ← new Student(101, "Alice")
  if Student.isEqual(student1, student2) then
    Print "Both student objects are equal."
  else
    Print "Student objects are not equal."
```

## ❒ Code:-

```
    class Student {
    int rollNo;
    String name;
    // Parameterized constructor
    public Student(int rollNo, String name) {
        this.rollNo = rollNo;
        this.name = name;
    }
    // Static method to check equality of two Student objects
    public static boolean isEqual(Student s1, Student s2) {
        return (s1.rollNo == s2.rollNo) &&
        (s1.name.equals(s2.name));
    }
}
public class StudentDemo {
    public static void main(String[] args) {
        // Creating two student objects
        Student student1 = new Student(101, "Alice");
        Student student2 = new Student(101, "Alice");
```

```
        // Check if students are equal
        if (Student.isEqual(student1, student2)) {
        System.out.println("Both student objects are equal.");
        } else {
        System.out.println("Student objects are not equal.");
        }
    }
}
```

❒ **Output:-**

**Figure 22**

❒ **Conclusion:-** The Java program demonstrates object comparison by defining a static method `isEqual()` in the `Student` class. It compares two `Student` objects based on both roll number and name, illustrating how logical equality can be implemented beyond reference equality. This exercise reinforces the use of parameterized constructors, static utility methods, and fundamental principles of encapsulation and data comparison in object-oriented programming.

## Problem Statement-23:-

**Create an abstract class employee, having its properties and abstract function for calculating net salary and displaying the information. Derive manager and clerk class from this abstract class and implement the abstract method net salary and override the display method.**

### ❒ Objective:-

- **Understanding Abstract Classes and Methods:** To learn how to define and use abstract classes and abstract methods in Java for enforcing method implementation in derived classes.
- **Practicing Inheritance and Method Overriding:** To implement inheritance by creating subclasses that extend an abstract class and override its methods.
- **Implementing Polymorphic Behavior:** To use dynamic method dispatch by invoking overridden methods using base class references.
- **Designing Role-Specific Salary Calculation:** To define and implement different logic for calculating net salary based on employee type (Manager or Clerk).
- **Enhancing Data Encapsulation and Output Clarity:** To encapsulate employee properties and ensure well-formatted display of employee details using overridden methods.

## ❒ Algorithm:-

**Algorithm 27** Employee Salary Calculation Using Abstract Class

```
Abstract Class Employee
    Variables: name, id, basicSalary
    Constructor Employee(name, id, basicSalary)
        Initialize instance variables
    Abstract Method: calculateNetSalary()
    Method: display()
        Print employee ID, name, and basic salary

Class Manager extends Employee
    Constructor: calls superclass constructor
    Method: calculateNetSalary()
        allowance ← 20% of basicSalary
        deduction ← 10% of basicSalary
        return basicSalary + allowance - deduction
    Method: display()
        Call super.display()
        Print position and net salary

Class Clerk extends Employee
    Constructor: calls superclass constructor
    Method: calculateNetSalary()
        allowance ← 10% of basicSalary
        deduction ← 5% of basicSalary
        return basicSalary + allowance - deduction
    Method: display()
        Call super.display()
        Print position and net salary

Function: main()
    emp1 ← new Manager("Alice", 101, 50000)
    emp2 ← new Clerk("Bob", 102, 30000)
    Print "=== Manager Details ==="
    emp1.display()
    Print "=== Clerk Details ==="
    emp2.display()
```

## ❒ Code:-

```
    // Abstract class
abstract class Employee {
    String name;
    int id;
    double basicSalary;
    public Employee(String name, int id, double basicSalary) {
        this.name = name;
```

```
        this.id = id;
        this.basicSalary = basicSalary;
    }
    // Abstract method to calculate net salary
    abstract double calculateNetSalary();
    // Method to display employee details
    public void display() {
        System.out.println("Employee ID: " + id);
        System.out.println("Name: " + name);
        System.out.println("Basic Salary: " + basicSalary);
    }
}
// Manager class
class Manager extends Employee {
    public Manager(String name, int id, double basicSalary) {
        super(name, id, basicSalary);
    }
    @Override
    double calculateNetSalary() {
        // Example: 20% allowance and 10% deduction
        double allowance = 0.20 * basicSalary;
        double deduction = 0.10 * basicSalary;
        return basicSalary + allowance - deduction;
    }
    @Override
    public void display() {
        super.display();
        System.out.println("Position: Manager");
        System.out.println("Net Salary: " + calculateNetSalary());
    }
}
// Clerk class
class Clerk extends Employee {
    public Clerk(String name, int id, double basicSalary) {
        super(name, id, basicSalary);
    }
```

```
    @Override
    double calculateNetSalary() {
        // Example: 10% allowance and 5% deduction
        double allowance = 0.10 * basicSalary;
        double deduction = 0.05 * basicSalary;
        return basicSalary + allowance - deduction;
    }
    @Override
    public void display() {
        super.display();
        System.out.println("Position: Clerk");
        System.out.println("Net Salary: " + calculateNetSalary());
    }
}
// Main class
public class EmployeeDemo {
    public static void main(String[] args) {
        Employee emp1 = new Manager("Alice", 101, 50000);
        Employee emp2 = new Clerk("Bob", 102, 30000);

        System.out.println("=== Manager Details ===");
        emp1.display();

        System.out.println("\n=== Clerk Details ===");
        emp2.display();
    }
}
```

## ❒ Output:-

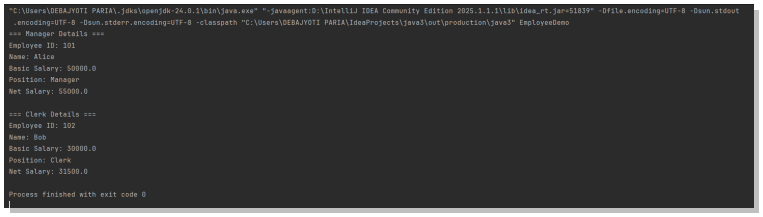



**Figure 23**

- **Conclusion:-** This Java program illustrates the use of abstract classes and method overriding in an employee management scenario. The abstract class `Employee` defines a base structure with a constructor, an abstract method `calculateNetSalary()`, and a concrete `display()` method. Subclasses `Manager` and `Clerk` override the abstract method to provide their own net salary calculations using different allowance and deduction percentages. This demonstrates runtime polymorphism, abstraction, and clean class hierarchy design.

# Acknowledgment

It is with a deep sense of gratitude and respect that I take this opportunity to express my heartfelt appreciation to all those who have contributed to the successful completion of this Java assignment.

First and foremost, I would like to express my sincere thanks and profound gratitude to Dr. Sumana Bandopadhaya, our esteemed Java teacher, for her invaluable support, continuous encouragement, and insightful guidance throughout the course. Her clear explanation of concepts, patient teaching, and commitment to student success have played a vital role in enhancing my understanding of Java programming. Her motivation and mentorship inspired me to explore the subject more deeply and complete this assignment with dedication and clarity.

I would also like to extend my gratitude to the school/college administration for providing a positive academic environment and the necessary infrastructure to carry out this project effectively.

Lastly, I would like to thank my family for their constant support, patience, and motivation throughout this journey. Their understanding and encouragement helped me stay focused and complete this assignment on time.

This assignment has not only helped me strengthen my programming skills but also allowed me to appreciate the value of consistent effort, discipline, and collaborative learning.

Thank you,

-.-.-.